

SignVerse AI

An AI-Powered Intelligent Sign Language
Learning & Communication Platform

Report Type	Engineering Project Report — AI/ML Track
Focus Areas	Computer Vision, Machine Learning, Accessibility Tech
Build Window	10-Day Rapid Development Sprint
Target Language	Indian Sign Language (ISL) — Phase 1
Prepared For	Expo / Hackathon Submission

Table of Contents

1. Abstract
2. Objectives
3. Existing System — Merits & Demerits
4. Proposed System (SignVerse AI) — Merits & Demerits
5. Complete System Architecture
6. Technology Stack
7. Core AI/ML Pipeline (Gesture Recognition)
8. Database Schema Overview
9. 10-Day Development Sprint Plan
10. Winning Enhancements for Expo / Hackathon
11. Risk Mitigation & Fallback Plan
12. Conclusion

1. Abstract

Communication barriers between deaf or hard-of-hearing individuals and the hearing community continue to limit accessibility in education, healthcare, workplaces, and everyday interactions. Although several sign language learning applications exist, most rely on static videos or predefined lessons and lack intelligent feedback, personalized learning, and real-time communication capabilities.

SignVerse AI is an Artificial Intelligence-powered platform designed to transform sign language learning into an interactive and personalized experience. Inspired by Duolingo's gamified learning approach, the platform integrates Computer Vision and Machine Learning to recognize hand gestures in real time, evaluate user accuracy, and provide instant corrective feedback. The system also offers adaptive learning paths, progress tracking, gamification, and AI-driven recommendations to improve learning efficiency.

Beyond education, the platform includes real-time speech-to-sign, sign-to-text, and text-to-speech translation features, enabling seamless communication between hearing and deaf individuals. By combining AI, accessibility, and human-centered design, SignVerse AI promotes inclusive communication while supporting the goals of Artificial Intelligence, Human-Machine Collaboration, and Industry 5.0. The proposed solution aims to create a socially impactful, scalable, and intelligent accessibility platform suitable for educational institutions, healthcare environments, public services, and everyday communication.

2. Objectives

Primary Objective

To develop an AI-powered sign language learning and communication platform that provides personalized learning, real-time gesture recognition, and intelligent communication assistance for both hearing and deaf communities.

Specific Objectives

- Develop a gamified learning platform inspired by Duolingo for Indian Sign Language (ISL).
- Implement AI-based real-time hand gesture recognition using Computer Vision.
- Provide instant feedback by evaluating the accuracy of user gestures.
- Generate personalized learning paths based on user performance using Machine Learning.
- Enable real-time Sign-to-Text, Speech-to-Sign, and Text-to-Speech communication.
- Track learning progress through analytics, achievements, and performance reports.
- Improve accessibility and promote inclusive communication in education, healthcare, and workplaces.
- Build a scalable platform that can support multiple sign languages in the future.

3. Existing System — Merits & Demerits

Current solutions in the sign-language space generally fall into three categories: static video-lesson apps (e.g. YouTube-based tutorials), dictionary/lookup apps (word-to-sign reference tools), and isolated academic gesture-recognition prototypes that are not deployed as usable products.

Aspect	Existing Systems
Learning Format	Static pre-recorded videos or PDF/image dictionaries with no interactivity.
Feedback Mechanism	None — the learner watches a sign but receives no evaluation of whether their own gesture was correct.
Personalization	One-size-fits-all curriculum; no adaptive difficulty or performance-based path.
Real-time Communication	Largely absent; most apps are learning-only, not communication tools.
Gamification	Minimal — few or no streaks, XP, levels, or motivational design.
Language Coverage	Heavily biased toward ASL (American Sign Language); ISL support is scarce.
Accessibility for Hearing Users	Rarely bridges both directions (deaf-to-hearing and hearing-to-deaf).

Merits of Existing Systems

- Low computational requirements since most are video-playback only.
- Simple to build and maintain; well-understood content-delivery model.
- Wide availability of ASL reference content due to larger existing user base.

Demerits of Existing Systems

- No way to verify if the learner is actually performing the sign correctly.
- Passive learning leads to poor retention and low engagement over time.
- No adaptive difficulty — advanced learners get bored, beginners get overwhelmed.
- Cannot be used as a live communication bridge between a deaf and hearing person.
- Little to no support for regional sign languages such as ISL.
- No analytics or progress tracking to show measurable improvement.

4. Proposed System (SignVerse AI) — Merits & Demerits

SignVerse AI closes the gaps above by combining a gamified adaptive-learning engine with real-time Computer Vision gesture recognition and a two-way communication translator.

Aspect	SignVerse AI (Proposed)
Learning Format	Interactive, camera-based lessons with live gesture practice, similar to Duolingo's exercise loop.
Feedback Mechanism	Real-time AI evaluation of hand landmarks with an on-screen accuracy score and correction hints.
Personalization	ML-driven adaptive learning path that adjusts difficulty and revisits weak signs.
Real-time Communication	Sign-to-Text, Speech-to-Sign, and Text-to-Speech modules for live two-way conversation.
Gamification	XP, streaks, levels, badges, and leaderboards to drive daily engagement.
Language Coverage	Indian Sign Language (ISL) first, architected to extend to other sign languages.
Accessibility Scope	Serves both deaf/hard-of-hearing users and hearing users, enabling bidirectional inclusion.

Merits of the Proposed System

- Active, verified learning — the AI confirms whether the gesture is actually correct.
- Higher engagement and retention through gamified, Duolingo-style mechanics.
- Personalized pacing keeps both beginners and advanced learners engaged.
- Doubles as a real-world communication aid, not just a learning tool.
- Fills the significant ISL content gap in the current app ecosystem.
- Progress analytics give learners, teachers, and institutions measurable outcomes.
- Modular architecture allows future expansion to more sign languages and use-cases.

Demerits / Challenges of the Proposed System

- Requires camera access and reasonable lighting for reliable gesture recognition.
- Higher computational cost than static-video apps (real-time CV inference).
- Building a labeled ISL gesture dataset is more effort than reusing existing ASL sets.
- Real-time recognition accuracy can vary across skin tones, backgrounds, and hand speeds — needs careful model tuning.
- Two-way translation (speech/text/sign) adds integration complexity within a short timeline.

5. Complete System Architecture

SignVerse AI follows a layered, modular architecture so each piece (learning engine, gesture recognition, translation, gamification) can be built and demoed independently, then wired together — ideal for a 10-day sprint.

5.1 High-Level Architecture Diagram (Layered View)

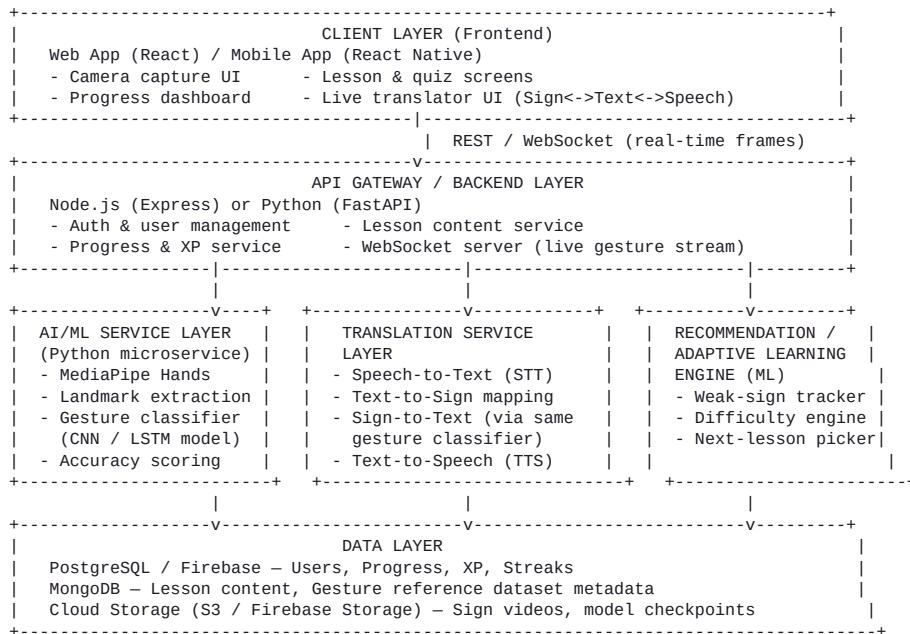


Figure 1: Layered system architecture of SignVerse AI.

5.2 Module Breakdown

Module	Responsibility
Client App	Captures camera frames, renders lessons/quizzes, shows real-time accuracy feedback and translator UI.
Backend API	Handles auth, lesson content delivery, XP/streak updates, and routes video frames to the AI service via WebSocket.
Gesture Recognition Engine	Uses MediaPipe to extract 21-point hand landmarks per frame, then a lightweight classifier (CNN/LSTM) predicts the sign and confidence score.
Translation Service	Chains STT -> Text -> Sign-animation mapping for Speech-to-Sign, and Gesture-classifier -> Text -> TTS for Sign-to-Speech.
Adaptive Learning Engine	Analyzes per-user accuracy history to decide the next lesson difficulty and which signs to re-drill.
Gamification Service	Manages XP, streaks, badges, and leaderboard rankings.
Data Layer	Stores user profiles, lesson content, progress logs, and trained model artifacts.

6. Technology Stack

Layer	Recommended Technology	Why
Frontend (Web)	React.js + Tailwind CSS	Fast to build, component reuse for lesson screens.
Frontend (Mobile, optional)	React Native / Flutter	Reuse logic; camera access on-device.
Backend API	FastAPI (Python) or Node.js + Express	FastAPI keeps AI + API in one language; fast to prototype.
Real-time Streaming	WebSockets (Socket.IO / native WS)	Low-latency frame/landmark streaming for live feedback.
Computer Vision	MediaPipe Hands / OpenCV	Pretrained, fast, runs on CPU — ideal for a 10-day build.
Gesture Classification	TensorFlow / Keras (CNN or LSTM on landmark sequences)	Landmark-based models train quickly on small datasets.
Speech-to-Text	Web Speech API / Google Cloud Speech-to-Text	Free browser API for quick demo; cloud API for accuracy.
Text-to-Speech	Web Speech API / gTTS	No-cost, quick integration.
Database	Firebase Firestore or PostgreSQL	Firebase speeds up auth + realtime sync for a hackathon timeline.
Storage	Firebase Storage / AWS S3	Store reference sign videos and model files.
Deployment	Vercel (frontend) + Render/Railway (backend)	Free-tier friendly, quick CI/CD for demo day.

7. Core AI/ML Pipeline (Gesture Recognition)

This is the technical heart of the project. The pipeline is designed to be trainable on a small, self-recorded ISL dataset within a few days.

Step 1 — Frame Capture

Camera feed captured in the browser/app and sent as frames or a live video stream.

Step 2 — Hand Landmark Detection

MediaPipe Hands extracts 21 (x, y, z) key points per hand per frame — no need to train a detector from scratch.

Step 3 — Feature Normalization

Landmark coordinates are normalized relative to the wrist point and scaled, making the model robust to hand position/size/distance from camera.

Step 4 — Sequence Modeling

For static signs, a simple feed-forward/CNN classifier on landmark vectors is enough. For motion-based signs, an LSTM/GRU processes a short sequence of frames to capture motion.

Step 5 — Classification & Confidence

The model outputs the predicted sign and a confidence score; low-confidence predictions trigger a 'try again' hint instead of a wrong-answer penalty.

Step 6 — Accuracy Scoring & Feedback

The confidence score is mapped to an on-screen accuracy percentage plus a specific correction tip (e.g., 'raise your thumb slightly').

Step 7 — Feedback Loop into Adaptive Engine

Each attempt (correct/incorrect, response time) is logged and used by the adaptive learning engine to schedule spaced repetition of weak signs.

8. Database Schema Overview

Collection / Table	Key Fields
Users	user_id, name, email, role (learner/teacher), created_at
Lessons	lesson_id, title, difficulty_level, sign_ids[], category
Signs (Reference Dataset)	sign_id, word/label, reference_video_url, landmark_template, language (ISL/ASL)
Progress	user_id, sign_id, attempts, best_accuracy, last_attempt_at, mastered (bool)
Gamification	user_id, xp, streak_count, badges[], level
Translator Logs (optional)	session_id, mode (speech2sign/sign2text), input, output, timestamp

9. 10-Day Development Sprint Plan

This plan assumes a small team (2-4 people) working in parallel tracks: Frontend, Backend/API, AI/ML, and Content/Design. Each day ends with a working, demoable increment so the project is never far from a presentable state.

Day	Focus	Key Tasks
Day 1	Planning & Setup	Finalize scope (ISL sign list ~30-50 words for MVP), set up repo, environments, Firebase/DB project, wireframe key screens.
Day 2	Data Collection	Record short reference videos for each MVP sign; run MediaPipe on them to build a landmark reference dataset.
Day 3	Core AI Model v1	Build and train the landmark-based gesture classifier; validate accuracy on a held-out test set.
Day 4	Backend Foundation	Set up FastAPI/Express backend, auth, user model, lesson content APIs; connect DB.
Day 5	Frontend Foundation	Build lesson list, lesson detail, and camera-capture screen in React; connect to backend APIs.
Day 6	Real-time Integration	Wire camera feed -> WebSocket -> AI service -> live accuracy feedback UI. This is the core demo moment — prioritize it.
Day 7	Gamification + Progress	Add XP, streaks, badges, and a progress dashboard with charts.
Day 8	Translator Module	Implement Speech-to-Sign (STT -> text -> sign video/animation playback) and Sign-to-Text/Speech (reuse classifier -> TTS).
Day 9	Adaptive Learning + Polish	Add simple adaptive logic (re-drill low-accuracy signs), fix bugs, polish UI/UX, add onboarding flow.
Day 10	Testing, Deployment & Pitch	End-to-end testing, deploy frontend + backend, rehearse demo script, prepare slides and this report.

Tip: Treat Day 6 (real-time gesture feedback) as the make-or-break milestone — it's the feature judges will remember most. If time runs short, cut translator polish before cutting this.

10. Winning Enhancements for Expo / Hackathon

These additions go beyond the core MVP and are designed to differentiate SignVerse AI in front of judges — pick 2-3 that fit your remaining time rather than attempting all.

- **AI Sign Coach (Pose Correction Overlay)**

Overlay skeletal hand-landmark guides directly on the user's camera feed showing exactly where their fingers should be, like a 'ghost hand' — a highly visual, memorable demo moment.

- **Emotion-Aware Avatar Interpreter**

A 3D/2D avatar that performs Speech-to-Sign translation with appropriate facial expression and pacing, making the translator feel more human and natural.

- **Offline-First Mode**

Run a lightweight on-device model (TensorFlow Lite / MediaPipe on-device) so core lessons and gesture checks work without internet — strong accessibility and real-world impact angle.

- **Multiplayer 'Sign Duel' Mode**

Two users compete live to sign a word fastest and most accurately, adding a social, Kahoot-style layer that's fun to demo on stage.

- **Teacher/Institution Dashboard**

An analytics view for teachers to track a whole class's ISL progress — broadens the pitch from 'an app' to 'a platform for schools.'

- **Voice + Facial Expression Recognition for Grammar**

Since sign language grammar uses facial expressions and mouth shapes as much as hand shape, add basic facial-landmark tracking to detect question/negation expressions — a technically impressive differentiator few student projects attempt.

- **Accessibility Mode for Public Services**

A simplified kiosk-style UI preset for hospitals/government counters, showing how the same engine solves a real institutional pain point — strengthens the social-impact story.

- **Explainable AI Confidence Meter**

Show **why** a gesture was marked wrong (e.g., which specific landmark was off) instead of a binary right/wrong — demonstrates deeper ML understanding to judges.

11. Risk Mitigation & Fallback Plan

Risk	Mitigation
Low model accuracy with small dataset	Start with a small MVP sign set (25-30 signs); use landmark-based features (not raw pixels) which train reliably on limited data.
Real-time latency issues	Run inference client-side where possible (MediaPipe + TFLite in-browser via TensorFlow.js) to avoid network round-trips.
Time overrun on translator module	Treat the translator as a stretch goal after Day 8; the learning + gesture-recognition core is the priority deliverable.
Demo-day camera/lighting failure	Record a backup demo video in advance as a fallback if live camera recognition misbehaves on stage.
Team bandwidth constraints	Use free-tier managed services (Firebase, Vercel) to cut infrastructure setup time.

12. Conclusion

SignVerse AI reframes sign language learning from passive video-watching into an active, AI-verified skill-building experience, while simultaneously functioning as a real-time communication bridge between deaf and hearing users. Its layered architecture keeps each module independently buildable within a 10-day sprint, and its enhancement roadmap — from pose-correction overlays to offline-first accessibility — gives it clear room to stand out at an expo. Most importantly, the project addresses a genuine and under-served accessibility gap, particularly for Indian Sign Language, giving it both technical depth and real social impact.